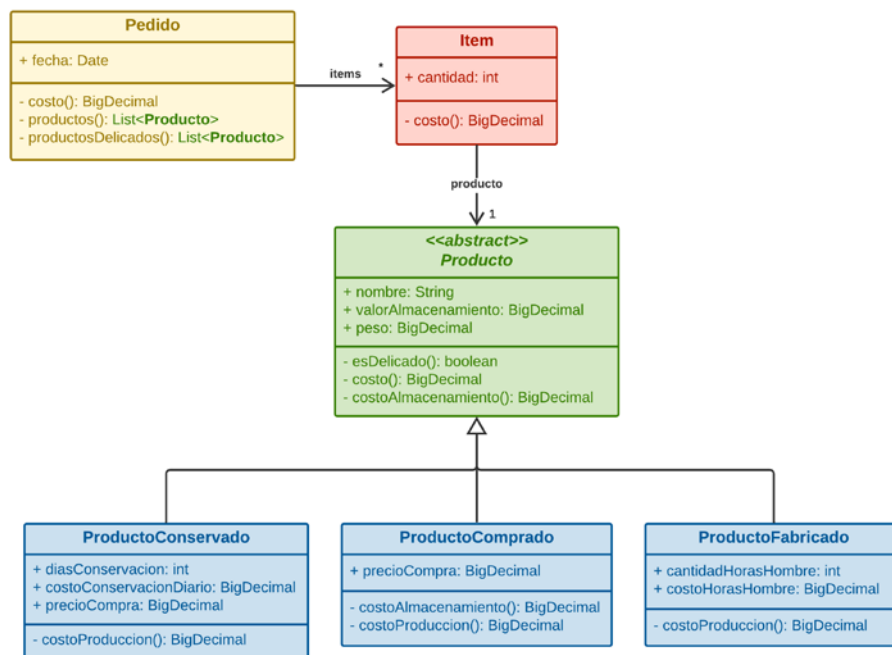


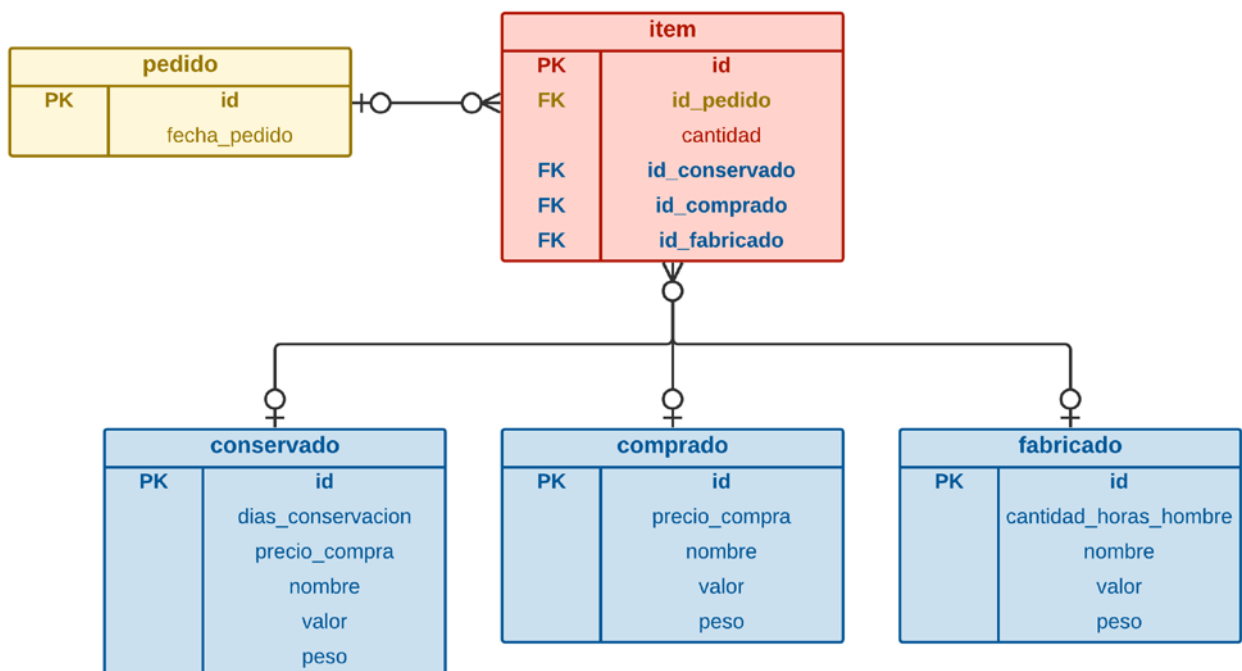
También llamada TABLE PER CLASS.

Tiene más sentido que se llame TABLE PER SUBCLASS (y no TABLE PER CLASS), ya que se mapea 1 tabla para cada **clase hija**...

- Conviene cuando:
  - o Las **clases hijas** tienen muy pocos atributos en común.
  - o Las **clases hijas** no tienen atributos en común.
- Permite trabajar con consultas SQL polimórficas.
- Las estructuras de datos resultan complejas.
- **BASE - Diagrama de Clases:**



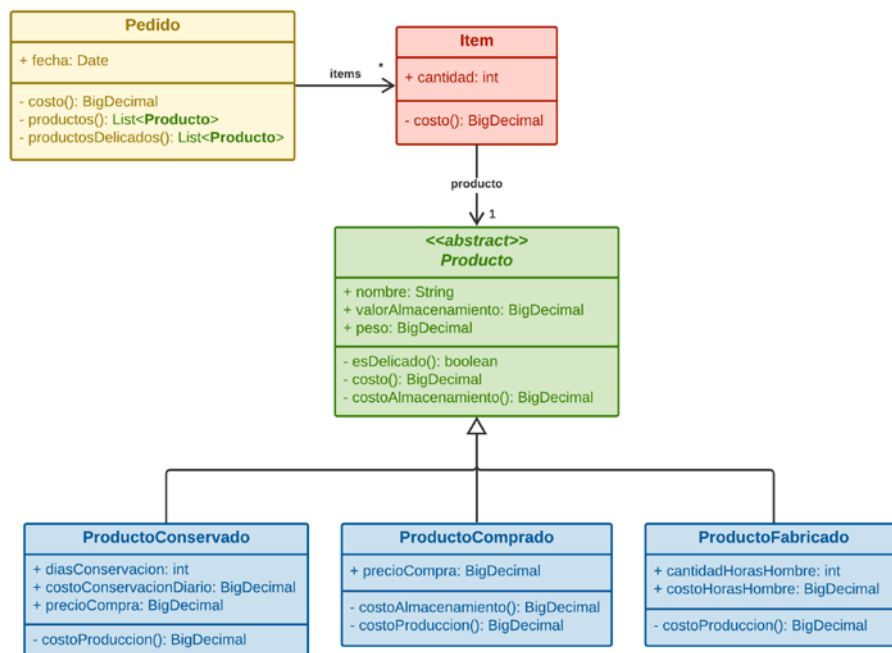
- **MAPEO:** Se persiste 1 tabla por cada **clase hija** → **N**.



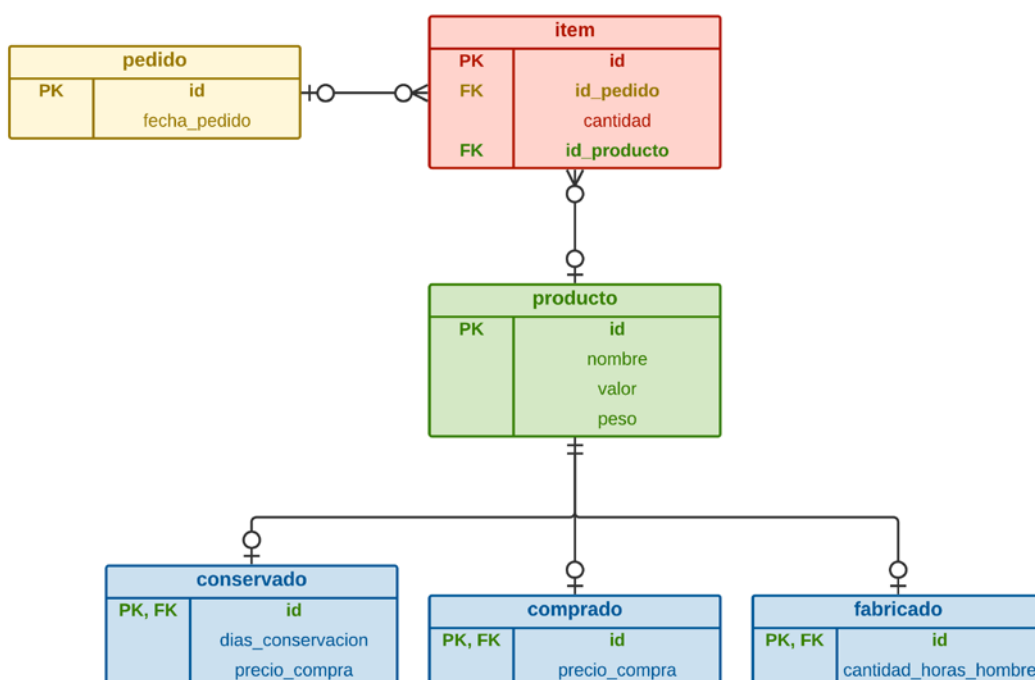
También llamada TABLE PER SUBCLASS.

Tiene más sentido que se llame TABLE PER CLASS (y no TABLE PER SUBCLASS), ya que se mapea 1 tabla por cada clase (una para la **clase padre** y una para cada una de las **clases hijas**)...

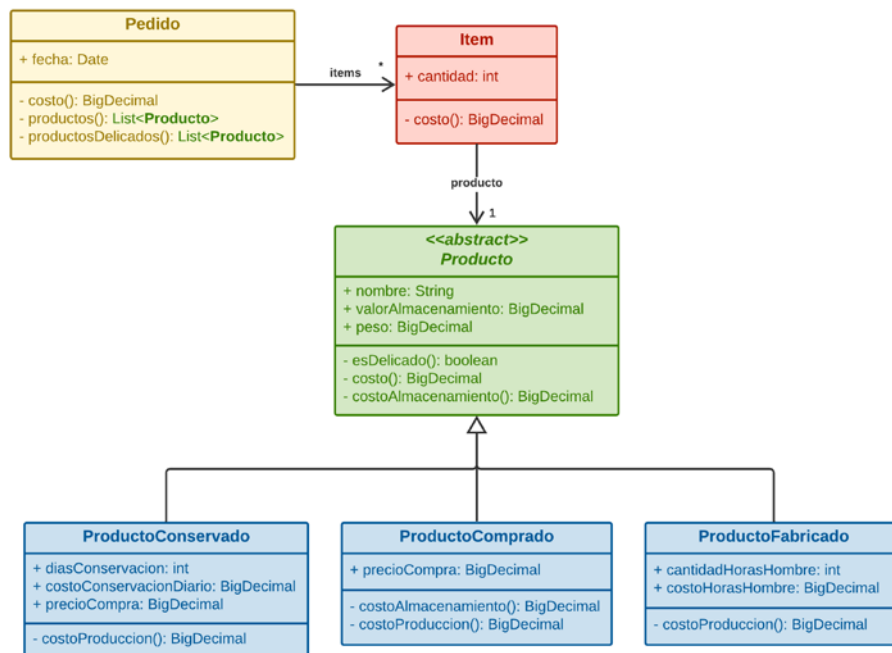
- Conviene cuando las **clases hijas** tienen muchos atributos en común y, al mismo tiempo, cada **clase hija** tiene atributos propios.
- Evita muchos NULLs (valores nulos) en los resultados de las consultas SQL.
- Me obliga a usar muchos JOINS en las consultas SQL, lo cual afecta la performance.
- **BASE - Diagrama de Clases:**



- **MAPEO:** Se persiste 1 tabla para la **clase padre** y 1 tabla por cada **clase hija** → **N + 1**.



- Conviene cuando las **clases hijas** tienen muchos atributos en común.
- Es simple → al generarse una única tabla, las consultas SQL son más simples.
- Tiene buena performance en general.
- Genera una tendencia al desorden de información.
- Los campos no utilizados deben aceptar NULLs (valores nulos).
- Necesita de un campo discriminador para poder reconstruir o “hidratar” el objeto.
- **BASE - Diagrama de Clases:**



- **MAPEO:** Se persiste 1 única tabla (una **clase padre** pero modificada) → **1**.

